

Design and Implementation of a Real Time Data Platform for the Stock Market Integrating Anomaly Detection and MLOps

Ho Duy Truong, Truong Vinh Linh

Abstract—This paper presents the design and implementation of a real time data platform for stock market applications, built on modern data engineering principles and machine learning lifecycle management. In financial markets, data is generated continuously, changes rapidly, and requires low latency processing. Traditional batch processing systems therefore face limitations in providing timely information for market monitoring and analysis. The proposed architecture follows an event driven design, in which stock price data is collected from external sources, transmitted through Apache Kafka, processed by Apache Flink and dedicated workers, and then stored in ClickHouse and ScyllaDB to support both analytical queries and low-latency access. In addition to the stock price data flow, the system integrates financial news collection, sentiment analysis, anomaly detection using Bayesian Online Change Point Detection, and a Whale ML model for forecasting trends in subsequent trading sessions after significant market movements. The processes of model training, experiment tracking, model version management, and inference serving are standardized according to MLOps practices through Airflow, MLflow, and MinIO. Experimental deployment results show that the system can process near real time data, provide APIs and visual dashboards, detect abnormal signals, and support predictive model operation within a unified architecture.

Index Terms—Apache Flink, Apache Kafka, Bayesian Online Change Point Detection, ClickHouse, Data Engineering, real time data, MLOps, ScyllaDB, stock market.

I. INTRODUCTION

IN recent years, the stock market has played an increasingly important role in the economy and has attracted strong participation from individual investors, financial institutions, securities companies, and online trading platforms. The development of digital infrastructure has increased the frequency and volume of market data, including price data, trading volume, historical records, financial news, market indices, and abnormal signals generated during trading activities. A key characteristic of stock market data is its time sensitivity. Information related to prices, trading volume, or financial news may be highly valuable at the time it appears, but its usefulness can decline quickly if it is processed late or is not incorporated into an analytical system at the right time.

This manuscript is submitted to Student Scientific Research Communication, Volume II, 2025. This research was conducted at the Faculty of Information Technology, Industrial University of Ho Chi Minh City.

H. D. Truong is with the Faculty of Information Technology, Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam (e-mail: hoduytruong280220@gmail.com).

T. V. Linh is with the Faculty of Information Technology, Industrial University of Ho Chi Minh City, Ho Chi Minh City, Vietnam (e-mail: truongvinhlh@iuh.edu.vn).

In many traditional systems, data is usually processed in batches. This model is suitable for end of day reports, periodic summaries, or historical analysis, but it does not sufficiently support applications that require continuous updates [10]. In the stock market context, delays in collecting, processing, and displaying data may reduce the ability to identify important market movements at an early stage. Therefore, a real time data platform must not only collect data quickly but also support stream processing, appropriate storage, API delivery, dashboard visualization, and advanced analytical modules.

In addition to displaying the latest data, modern financial analytics systems also require the ability to detect abnormal events and manage machine learning models after deployment. Stock market data is noisy, its distribution changes over time, and it is affected by many external factors. Therefore, forecasting models can quickly degrade if they are not monitored, evaluated, and updated periodically [12]. This is the motivation for integrating MLOps principles into the system to standardize model training, evaluation, storage, deployment, and monitoring.

This study proposes a real time data platform architecture for the stock market by combining data pipelines, anomaly detection, news sentiment analysis, and machine learning models. The main contributions of this paper are as follows: (1) designing an event-driven data architecture using Kafka [2], Flink [3], ClickHouse [4], and ScyllaDB [5]; (2) integrating a Bayesian Online Change Point Detection (BOCPD) module [1] to identify state transition points; (3) developing a Whale ML module to forecast market trends after abnormal events; and (4) organizing an MLOps workflow with Airflow [6], MLflow [7], and MinIO to support model lifecycle management.

The remainder of this paper is organized as follows. Section II presents the related background and research problem. Section III describes the system requirements and overall architecture. Section IV details the design and implementation of the main subsystems. Section V discusses anomaly analysis, forecasting, and MLOps. Section VI presents experimental results and discussion. Section VII describes limitations, future work, and the conclusion.

II. RELATED BACKGROUND AND RESEARCH PROBLEM

A. Real Time Data in the Stock Market

Real time data refers to data that is generated, transmitted, and processed almost immediately after it occurs. Unlike batch

processed data, real time data is usually handled continuously as a stream of events [10]. In the stock market domain, data can come from different sources such as price APIs, transaction data, OHLCV data, financial news, and market status information [9]. These data sources are heterogeneous in terms of format, update frequency, and reliability. Therefore, an ingestion and standardization layer is required before the data is transferred to downstream processing components.

A real time data platform for stock market applications must satisfy several requirements at the same time. First, the system needs low latency so that the latest data can be displayed or utilized quickly. Second, the system must be scalable when the number of stock symbols or the data update frequency increases. Third, data must be stored for different purposes, including latest state data for dashboards, historical data for analysis, feature data for model training, and alert data for market monitoring.

B. Stream Processing and Event Driven Architecture

An event-driven architecture enables separation between data producers and data consumers. Instead of allowing each service to connect directly to the original data source, all records are sent to a message broker. In this study, Apache Kafka [2] is used as the central data transmission layer. Kafka organizes data into topics and partitions, allowing multiple consumers to process data in parallel while reducing direct dependencies among the ingest service, processing workers, backend, and analytical modules.

After the transmission layer, Apache Flink [3] is used for stateful stream processing. Flink is suitable for tasks such as data cleaning, record normalization, time window aggregation, and OHLCV generation. Compared with other large scale processing engines such as Apache Spark [13], Flink emphasizes low latency stream processing, which fits the requirements of this system. Window operations such as tumbling windows and sliding windows help transform tick level or continuously updated data into more structured time series data, which is convenient for candlestick charts and machine learning models.

C. Multi Tier Storage

A stock market data system should not rely on a single type of database for all purposes. Historical data usually requires queries by stock symbol, time range, and large scale aggregation, while the latest data requires fast responses with low latency. Therefore, this paper adopts a multi tier storage model. ClickHouse [4] is used for large scale analytical data, historical queries, and training feature generation. ScyllaDB [5] is used for the latest states, dashboard serving data, recent prediction results, and real time alerts.

D. Anomaly Detection and Model Management

Anomaly detection in stock time series can be considered the task of identifying moments when the data distribution changes significantly [11]. BOCPD [1] is suitable for online data because the algorithm updates the changepoint probability

whenever a new observation arrives. When the probability of a state change exceeds a predefined threshold, the system can generate an alert for users or use this signal as an input feature for forecasting models.

However, financial forecasting models are often affected by data drift, changes in market regimes, and noise from news events [12]. Therefore, MLOps is integrated to manage the processes of training, evaluation, model registration, and model serving. Airflow [6] orchestrates scheduled training pipelines, MLflow [7] records parameters and experimental metrics, and MinIO stores model artifacts.

III. SYSTEM REQUIREMENTS AND PROPOSED ARCHITECTURE

A. Functional Requirements

The system is designed to satisfy several major functional groups. The first group involves collecting and transmitting real time stock price data. Input data must be validated, standardized, and published to Kafka so that other components can consume it independently. The second group involves stream processing, including data cleaning, normalization, OHLCV aggregation, and storage of processed data. The third group includes extended analytics, such as news collection, sentiment analysis, anomaly detection, and trend forecasting. The fourth group provides data to users through backend APIs, WebSocket, and dashboards. The final group supports operational monitoring, including service status, latency, error logs, resource usage, and model status.

B. Non Functional Requirements

In addition to functional requirements, the system must satisfy several non functional requirements. Low latency is the most important requirement because stock market data has high time value. Stability must be ensured when multiple components such as the ingest service, Kafka, Flink, databases, backend, and analytical workers operate simultaneously. Scalability must be considered from the beginning through Kafka partitioning, Flink parallelism, and scalable storage design. Furthermore, the system should be easy to maintain, easy to monitor, and capable of replacing individual services without affecting the entire pipeline.

C. Overall Architecture

The proposed system is organized into six main layers: the data ingestion layer, data transmission layer, stream processing layer, storage layer, machine learning analytics layer, and data serving layer. The data flow begins from external data sources, passes through the ingest service for standardization, and is then sent to Kafka. From Kafka, Flink and dedicated workers read the data for processing, aggregation, and enrichment. The processed data is written to ClickHouse and ScyllaDB. The backend API retrieves data from the storage layers to provide information to the dashboard, while the BOCPD, sentiment analysis, and Whale ML modules generate additional value added information.

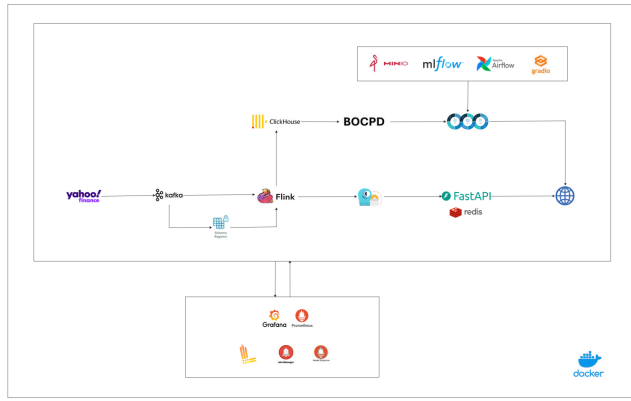


Fig. 1. Overall architecture of the real time data platform integrating anomaly detection and MLOps.

TABLE I
MAIN COMPONENTS IN THE SYSTEM ARCHITECTURE

Component	Main Role	Main Output
Ingest Service	Collects, validates, and standardizes input data	Standardized records
Apache Kafka	Transmits data using a streaming based model	Data topics for consumers
Apache Flink	Processes streams and performs time window aggregation	OHLCV data
ClickHouse	Stores historical and analytical data	OLAP queries and ML features
ScyllaDB	Stores latest states and low latency serving data	Realtime API and dashboard data
BOCPD Module	Detects state transition points	Anomaly signals
Whale ML	Trains models and performs trend inference	Forecasting results
Backend API	Aggregates data from multiple sources	REST API and Web-Socket
Monitoring Stack	Monitors operational status	Metrics, logs, and alerts

IV. DESIGN AND IMPLEMENTATION OF SUBSYSTEMS

A. Data Ingestion Subsystem

The data ingestion subsystem is the entry point of the entire pipeline. This component is responsible for connecting to external stock market data sources [9], receiving new data, validating record structures, handling missing or incorrectly formatted fields, and converting records into a unified schema. A standardized price record usually includes the stock symbol, event time, price, trading volume, data source, and system ingestion time.

To improve stability, the ingest service is designed as an independent worker with an automatic reconnection mechanism when the data source is interrupted. If data arrives irregularly, the service records logs and stale states to support monitoring. After standardization, data is sent to the corresponding Kafka topic, such as a real time price topic or a financial news topic. This organization ensures that downstream components do not directly depend on external data sources.

B. Kafka Topic Design

Kafka [2] is used as the backbone of data transmission. Topics are organized according to data type and usage purpose,

TABLE II
KAFKA TOPIC DESIGN IN THE SYSTEM

Topic	Main Consumer	Purpose
stock price realtime	Flink, storage worker	Real time stock price data
stock ohlcv	Backend, BOCPD	Window based candlestick data
stock news	Sentiment worker	Financial news by stock symbol
news sentiment	Backend, storage worker	News sentiment results
stock changepoint	Whale ML, dashboard	Anomaly signals
stock feature ml	Whale ML	Features for training and inference
stock prediction	Backend, dashboard	Trend forecasting results
system monitoring	Monitoring stack	Operational metrics and logs

allowing each consumer to subscribe to the exact data stream it needs. The real time price topic is the main input for Flink. The OHLCV topic contains aggregated data. News, sentiment, changepoint, and prediction topics serve analytical modules and dashboards.

C. Stream Processing with Flink and Dedicated Workers

Flink [3] handles stateful stream processing tasks. When price data is sent to Kafka, Flink reads the data through a consumer group and applies cleaning steps such as removing records with missing symbols, invalid timestamps, or invalid values. The data is then standardized to the same timezone, price unit, and schema structure.

The next important step is time-window aggregation to generate OHLCV data [10]. The opening price is taken from the first record in the window, the highest and lowest prices are calculated across the entire window, the closing price is taken from the final record, and the trading volume is accumulated. The generated OHLCV data is then written to a new topic or directly to the storage layer. For specialized tasks such as news analysis, anomaly detection, or feature generation for machine learning, the system uses additional dedicated workers to provide greater deployment flexibility.

D. ClickHouse and ScyllaDB Storage Design

ClickHouse [4] is used for historical data tables such as stock prices, OHLCV data, changepoint events, and features for model training. These tables are designed for queries by stock symbol and time range, making them suitable for retrieving price history over a period, aggregating daily data, or extracting training data for Whale ML.

ScyllaDB [5] is used for data that requires fast retrieval, such as the latest price of each stock symbol, recent news, latest sentiment results, anomaly alerts, and recent predictions. For dashboard queries, the backend often needs to respond quickly to users; therefore, ScyllaDB is suitable as a low latency serving layer. In some cases, important data can be written to both storage systems: ClickHouse stores long-term history, while ScyllaDB stores the latest state.

TABLE III
STORAGE ROLE ALLOCATION

Data Type	ClickHouse	ScyllaDB
Historical prices	Preferred	Not prioritized
OHLCV	Preferred	Reduced version if needed
Latest prices	Not prioritized	Preferred
Financial news	Long term analytical storage	Fast retrieval
Sentiment	Historical storage	Latest results
Changepoint	Event history storage	Realtime alerts
Whale ML features	Preferred	Not prioritized
Prediction	Forecasting history	Latest results

E. Backend API and Dashboard

The backend API acts as an intermediate layer between the data system and end users. This component retrieves data from ClickHouse and ScyllaDB, combines multiple data types such as prices, news, sentiment, changepoints, and predictions, and then returns results through REST APIs or WebSocket. REST APIs are suitable for historical or on demand queries, while WebSocket is suitable for real time updates on the dashboard.

The dashboard is designed to display major information groups, including stock symbol lists, latest prices, historical price charts, OHLCV candlestick charts, related news, sentiment analysis results, anomaly alerts, and trend forecasts. In addition, the operational dashboard displays the status of containers, Kafka, APIs, databases, Airflow, MLflow, and system resources.

V. ANOMALY ANALYSIS, FORECASTING, AND MLOPS

A. Sentiment Analysis from Financial News

Financial news is an important supplementary data source because price movements are often affected by corporate events, macroeconomic news, or market sentiment. The system collects news through RSS or financial data sources [9] and then maps each news item to related stock symbols. Each news record stores fields such as title, short description, link, publication time, source, and stock symbol.

After collection, the text is preprocessed and passed into a sentiment analysis model based on FinBERT [8]. The output includes a positive, negative, or neutral label together with a sentiment score. This data is stored for dashboard visualization and can also be used as a feature for forecasting models. Combining price data and news data allows the system not only to recognize market movements but also to provide context for explaining those movements.

B. Anomaly Detection Using BOCPD

BOCPD [1] is used to detect changepoints in price series or feature series derived from OHLCV data. Intuitively, the algorithm tracks the probability that the current observation sequence still belongs to the same movement regime or has just shifted to a new regime. When new data appears, the algorithm updates the run length distribution, which represents

the length of the observation sequence since the most recent changepoint.

In the system, BOCPD inputs may include returns, volatility, volume changes, or a combined feature set [11]. When the changepoint probability exceeds a configured threshold, the system generates an anomaly event containing the stock symbol, timestamp, change probability, anomaly level, and several related features. This event is written to Kafka and stored in ClickHouse or ScyllaDB. The dashboard can display the alert, while Whale ML uses the signal to forecast the next trend.

C. Whale ML for Trend Forecasting

Whale ML is a machine learning subsystem designed to forecast the trend of subsequent trading sessions after an abnormal event [12]. The goal of this subsystem is not to replace investment decisions, but to provide an additional analytical signal based on historical data, changepoints, and sentiment information. Model inputs include changepoint probability, anomaly score, predicted volatility, return values, price, volume, time related information, and market category variables.

The training workflow includes extracting data from ClickHouse, constructing the feature set, defining forecasting labels according to the selected horizon, splitting the dataset into training and testing sets, training a classification model, evaluating the model, and registering the model. Deep learning frameworks such as PyTorch [14] can be used for more advanced model variants. The system also supports both global models and symbol specific models. A global model uses data from multiple stock symbols to increase the number of samples, while a symbol specific model can learn the unique characteristics of a particular stock if sufficient data is available.

The Whale ML service is deployed as an independent API. Its main endpoints include model health checking, model information retrieval, global model training, symbol specific model training, single-event forecasting, and batch forecasting. When an inference request is received, the service loads the currently served model, standardizes the input features, and returns the predicted label or trend probability.

D. Model Lifecycle Management with MLOps

MLOps helps the system move from experimental models to operational models. In this study, Airflow [6] is used to orchestrate scheduled training pipelines. A training DAG may include data validation, feature extraction, model training, evaluation, comparison with the current model, and registration of a new model if it satisfies the predefined criteria.

MLflow [7] is used for experiment tracking. Each training run records information such as parameters, datasets, algorithms, evaluation metrics, training time, and model artifacts. MinIO acts as artifact storage to ensure that models can be reloaded when the service starts or when rollback is required. This organization makes model management more transparent, reduces the risk of using the wrong version, and supports the reproducibility of experimental results.

TABLE IV
MAIN API GROUPS OF THE SYSTEM

API Group	Example Endpoint	Purpose
Latest prices	/stocks/latest	Displays the current market state
Price history	/stocks/history/{symbol}	Draws historical price charts
OHLCV	/stocks/ohlc/{symbol}	Provides candlestick chart data
News	/news/{symbol}	Lists related news
Sentiment	/sentiment/{symbol}	Provides news sentiment results
Changepoint	/changepoint/{symbol}	Provides anomaly alerts
Prediction	/prediction/{symbol}	Provides forecasting results
Health	/system/health	Reports system status
WebSocket	/ws/market	Provides real time updates

TABLE V
EVALUATION CRITERIA AND QUALITATIVE RESULTS

Criterion	Observed Result	Significance
Pipeline latency	Achieves near real time processing	Suitable for dashboards
System stability	Services operate independently	Easier fault isolation
Scalability	Kafka partitioning and parallel workers	Supports workload expansion
Historical queries	ClickHouse performs effectively	Suitable for analytics
Fast retrieval	ScyllaDB serves the latest data	Suitable for realtime APIs
Anomaly analysis	BOCPD generates changepoint signals	Supports alerting
MLOps	Airflow, MLflow, and MinIO operate together	Improves model management
Monitoring	Metrics, logs, and service states are collected	Supports system operation

VI. EXPERIMENTAL RESULTS AND DISCUSSION

A. Experimental Deployment Environment

The system is deployed in a containerized environment, where each major component is separated into an individual service. This deployment approach makes dependency management easier, improves log tracking, and supports future scalability. The core services include the ingest service, Kafka [2], Flink [3] or processing workers, ClickHouse [4], ScyllaDB [5], backend API, dashboard, Whale ML service, Airflow [6], MLflow [7], MinIO, and the monitoring stack.

During testing, the system operates with a price data stream based on a predefined list of stock symbols. Data flows through the pipeline from ingestion to Kafka, processing, storage, and dashboard visualization. The BOCPD and Whale ML modules are tested using OHLCV data and anomaly events generated during system operation.

B. Deployment Results

The deployment results show that the pipeline can operate continuously from data collection to visualization. Kafka decouples system components, allowing multiple consumers to process the same data source without directly affecting the ingest service. Flink and processing workers effectively perform normalization, aggregation, and data preparation for analytical tasks.

ClickHouse demonstrates suitability for historical and aggregated data queries. Queries by stock symbol and time range can support historical dashboards and training feature extraction. ScyllaDB is suitable for data that requires fast retrieval, such as latest prices, recent news, alerts, and prediction results. The backend API combines the two storage layers to return dashboard data according to different usage purposes.

The BOCPD subsystem [1] generates changepoint signals when notable fluctuations appear in the data series. These signals are displayed on the dashboard and are also used as input for Whale ML. The Whale ML subsystem can train models, store models, load the currently served model, and return forecasting results through APIs. Airflow and MLflow support training pipeline management, experiment tracking, and model artifact storage.

C. Discussion

The experimental results show the value of combining Data Engineering and MLOps in a single system. Data Engineering provides the foundation for stable data collection, processing, storage, and serving [10]. MLOps allows machine learning models to move beyond experimentation so that they can be monitored, updated, and served through APIs.

One important observation is that the multi tier storage design helps avoid conflicts between analytical workloads and fast retrieval requirements. If only one database were used, the system could face difficulties when processing large historical queries while also responding to real time dashboard requests. Separating ClickHouse [4] and ScyllaDB [5] allows each layer to perform the role for which it is most suitable.

In addition, using Kafka [2] as the central transmission layer makes the system easier to extend in a modular way. When a new worker needs to be added, such as a social media analysis worker or a risk alert worker, the new service can subscribe to existing topics without modifying the ingest service. This is important for a data platform that is intended to support long term expansion.

VII. LIMITATIONS AND FUTURE WORK

Although the system has achieved the objective of building a real time data platform at the experimental level, several limitations remain. First, the scale of data and the number of monitored stock symbols are still limited and do not fully reflect high load enterprise environments. Second, the input data mainly focuses on prices, OHLCV, and news, while order book data, detailed transaction data, and macroeconomic data have not yet been deeply integrated. Third, the forecasting model currently serves as an analytical support tool and should not be considered an investment recommendation system. In financial markets, model effectiveness strongly depends on data quality, feature design, forecasting horizon, and market conditions [12].

In the future, the system can be extended in several directions. From the data perspective, more news sources, social media data, financial reports, and macroeconomic datasets can be added. From the modeling perspective, deep learning

methods for time series or hybrid models combining price and text data can be explored [14]. From the infrastructure perspective, the system can be deployed on Kubernetes to support auto scaling, rolling updates, and better service management. From the MLOps perspective, data testing, model testing, drift monitoring, and CI/CD workflows for both backend services and machine learning models can be added.

VIII. CONCLUSION

This paper presented the design and implementation of a real time data platform for the stock market, integrating data ingestion, data transmission, stream processing, multi tier storage, anomaly analysis, trend forecasting, and MLOps. The proposed architecture uses Kafka [2] to decouple data streams, Flink [3] and dedicated workers for stream processing, ClickHouse [4] for analytical storage, ScyllaDB [5] for fast data serving, BOCPD [1] for changepoint detection, and Whale ML to support trend forecasting after abnormal movements.

The deployment results show that the system can operate as a near real time pipeline, provide data through APIs and dashboards, and support advanced analytical modules. The combination of Data Engineering and MLOps allows the system not only to process data but also to establish a structured foundation for operating machine learning models. Although limitations remain in terms of data scale, model complexity, and deployment infrastructure, this study demonstrates the feasibility of building a modern data platform for monitoring, analyzing, and supporting decision making in the financial domain.

ACKNOWLEDGMENT

The authors sincerely thank the Faculty of Information Technology, Industrial University of Ho Chi Minh City, for providing academic support and facilities during this research. The authors also express their gratitude to the academic supervisor for guidance, feedback, and support throughout the development of this project.

REFERENCES

- [1] R. P. Adams and D. J. C. MacKay, "Bayesian online changepoint detection," *arXiv preprint arXiv:0710.3742*, 2007.
- [2] Apache Software Foundation, "Apache Kafka documentation," 2025. [Online]. Available: <https://kafka.apache.org/documentation/>. Accessed: May 1, 2025.
- [3] Apache Software Foundation, "Apache Flink documentation," 2025. [Online]. Available: <https://flink.apache.org/>. Accessed: May 1, 2025.
- [4] ClickHouse Inc., "ClickHouse documentation," 2025. [Online]. Available: <https://clickhouse.com/docs/>. Accessed: May 1, 2025.
- [5] ScyllaDB, "ScyllaDB documentation," 2025. [Online]. Available: <https://docs.scylladb.com/>. Accessed: May 1, 2025.
- [6] Apache Software Foundation, "Apache Airflow documentation," 2025. [Online]. Available: <https://airflow.apache.org/>. Accessed: May 1, 2025.
- [7] MLflow, "MLflow documentation," 2025. [Online]. Available: <https://mlflow.org/docs/latest/index.html>. Accessed: May 1, 2025.
- [8] ProsusAI, "FinBERT: Financial sentiment analysis with BERT," 2025. [Online]. Available: <https://huggingface.co/ProsusAI/finbert>. Accessed: May 1, 2025.
- [9] Yahoo Finance, "Yahoo Finance," 2025. [Online]. Available: <https://finance.yahoo.com>. Accessed: May 1, 2025.
- [10] T. Akidau, S. Chernyak, and R. Lax, *Streaming Systems: The What, Where, When, and How of Large-Scale Data Processing*. Sebastopol, CA, USA: O'Reilly Media, 2018.

- [11] F. I. Vazquez, D. Carrera, and D. Garcia-Gil, "Anomaly detection in streaming data: A comparison and evaluation study," *Expert Systems with Applications*, vol. 233, Art. no. 120959, 2023.
- [12] M. M. Kumbure, C. Lohrmann, P. Luukka, and J. Porras, "Machine learning techniques and data for stock market forecasting: A literature review," *Expert Systems with Applications*, vol. 197, Art. no. 116659, 2022.
- [13] M. Zaharia et al., "Apache Spark: A unified engine for big data processing," *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [14] A. Paszke et al., "PyTorch: An imperative style, high-performance deep learning library," in *Proc. NeurIPS*, 2019, pp. 8024–8035.